

Checkpoint 4

Indexing and Query Performance Analysis

PitchStats: Soccer Analytics Platform | CS 564

Aryan Garg, Jeremy Jarocki, Kaeya Kapoor

For this checkpoint we measured how indexes affect query performance on the fully loaded PitchStats database. All five queries were run against real Transfermarkt data, with the Appearance table holding 1,894,348 rows. We recorded each query's runtime and query plan before indexing, created four indexes, and then re-ran the same queries under the same conditions to compare.

1. Selected Queries for Performance Testing

We selected five queries from Checkpoint 3. Each involves large tables, multiple joins, filtering, grouping, or sorting, so none is a trivial single-table query.

Query 1: Young Attacking Talent Search

```
SELECT p.player_id, p.name, c.name AS club,
       ROUND(AVG(a.goals),3) AS avg_goals,
       COUNT(DISTINCT a.game_id) AS games_played
FROM Player p
LEFT JOIN Club c ON p.current_club_id = c.club_id
LEFT JOIN Appearance a ON p.player_id = a.player_id
WHERE (p.position = 'Attack' OR p.position = 'Midfield')
      AND p.date_of_birth >= '1995-01-01'
GROUP BY p.player_id, p.name, c.name
HAVING AVG(a.goals) > 0.5 AND COUNT(DISTINCT a.game_id) >= 50
ORDER BY avg_goals DESC, games_played DESC
LIMIT 20;
```

Purpose: Powers the scouting search on the Players/Clubs tab, finding young attacking players with strong scoring records.

Why it is a good candidate: Filters Player on two columns (position and date_of_birth) and joins to the 1.89M-row Appearance table, then groups and sorts. The filter columns had no index, so it was a good candidate.

Query 2: Player Market Value Trend

```
SELECT p.name, pv.date, pv.market_value_eur,
       LAG(pv.market_value_eur) OVER
         (PARTITION BY pv.player_id ORDER BY pv.date) AS prev
FROM PlayerValuation pv
JOIN Player p ON pv.player_id = p.player_id
WHERE p.name = 'Kylian Mbappé'
ORDER BY pv.date DESC
LIMIT 12;
```

Purpose: Drives the Graph tab, showing how a selected player's market value has moved over time using a window function.

Why it is a good candidate: The application looks a player up by name, but name has no index, forcing a full scan of the Player table on every lookup. This is exactly the kind of equality filter an index should fix.

Query 3: Club Performance by Competition

```
SELECT c.name, comp.name, COUNT(DISTINCT g.game_id) AS matches,
       SUM(CASE WHEN g.home_club_id = c.club_id AND g.home_score > g.away_score THEN 1
                WHEN g.away_club_id = c.club_id AND g.away_score > g.home_score THEN 1
                ELSE 0 END) AS wins
FROM Club c
JOIN Game g ON (g.home_club_id = c.club_id OR g.away_club_id = c.club_id)
JOIN Competition comp ON g.competition_id = comp.competition_id
```

```
WHERE c.name = 'Manchester City'
GROUP BY c.club_id, comp.competition_id, comp.name
ORDER BY matches DESC;
```

Purpose: Builds a club's win/draw/loss record per competition for the club dashboard.

Why it is a good candidate: The query filters Club by name, but with no index on name the optimizer scanned all clubs and used 'Range checked for each record' on the OR join, which is expensive. A name index should let it start from the single club.

Query 4: Top Scorers by Season

```
SELECT p.name, c.name AS club, SUM(a.goals) AS total_goals,
       ROUND(SUM(a.goals)*90.0/NULLIF(SUM(a.minutes_played),0),2) AS goals_per_90
FROM Player p
JOIN Appearance a ON p.player_id = a.player_id
JOIN Game g ON a.game_id = g.game_id
JOIN Club c ON a.club_id = c.club_id
WHERE g.season = 2024
GROUP BY p.player_id, p.name, c.name
HAVING SUM(a.goals) >= 20
ORDER BY total_goals DESC
LIMIT 15;
```

Purpose: The headline 'top scorers this season' feature, joining four tables and filtering by season.

Why it is a good candidate: Filters Game by season across a four-table join into the 1.89M-row Appearance table. We added an index on Game(season) expecting it to drive the query from the filtered games.

Query 5: Performance by Position

```
SELECT p.position, COUNT(DISTINCT p.player_id) AS player_count,
       SUM(a.goals) AS total_goals, ROUND(AVG(a.goals),4) AS avg_goals
FROM Player p
JOIN Appearance a ON p.player_id = a.player_id
WHERE a.minutes_played > 0
GROUP BY p.position
HAVING COUNT(DISTINCT p.player_id) >= 50
ORDER BY avg_goals DESC;
```

Purpose: A league-wide aggregate used on the Compare tab to show which positions are most productive.

Why it is a good candidate: This is the heaviest query in the application: it aggregates across all 1.89M appearances and groups by position. We added idx_player_position_dob to help the position grouping.

2. Baseline Performance Before Indexing

Each query was run three times and we reported the representative runtime. Runtimes were captured with MySQL query profiling (SET profiling = 1, then SHOW PROFILES). The database already held the full dataset, with the Appearance table at over 1.89 million rows.

Query 1: baseline

Runtime before indexing: 8.41 s

Query plan (EXPLAIN): Player is read with type = ALL (full table scan of ~50,000 rows) with Using where, Using temporary, Using filesort. It then joins to Appearance by the existing player_id key.

Query 2: baseline

Runtime before indexing: 0.011 s

Query plan (EXPLAIN): Player is read with type = ALL (full scan of ~50,000 rows) with Using where, then joined to PlayerValuation by player_id.

Query 3: baseline

Runtime before indexing: 0.65 s

Query plan (EXPLAIN): Competition is scanned (type = ALL), joined to Game by competition_id, and Club is read with type = ALL and 'Range checked for each record' because of the OR condition on home/away club.

Query 4: baseline

Runtime before indexing: 13.61 s

Query plan (EXPLAIN): Club is scanned (type = ALL) with Using temporary, Using filesort, then joined into Appearance by club_id and out to Player and Game by primary key. The season filter is applied late.

Query 5: baseline

Runtime before indexing: 57.60 s

Query plan (EXPLAIN): Player is read with type = ALL (full scan) with Using temporary, Using filesort, then joined to Appearance by player_id, scanning the matching appearance rows.

3. Index Creation

We created four indexes (the requirement is at least three). Each targets a column used for filtering, joining, or grouping in one or more of the selected queries.

Index 1: Player (position, date_of_birth)

```
CREATE INDEX idx_player_position_dob ON Player(position, date_of_birth);
```

Table and columns: Player (position, date_of_birth)

Query it helps: Queries 1 and 5

Why we chose it: Both queries filter or group on position, and Query 1 also filters on date_of_birth. A composite index on these two columns lets Query 1 do a range scan instead of a full table scan, and lets Query 5 read position values directly from the index.

Index 2: Player (name)

```
CREATE INDEX idx_player_name ON Player(name);
```

Table and columns: Player (name)

Query it helps: Query 2

Why we chose it: The application looks players up by name for the market-value graph. Without an index this is a full table scan on every lookup; with it, the lookup becomes a direct indexed access.

Index 3: Game (season)

```
CREATE INDEX idx_game_season ON Game(season);
```

Table and columns: Game (season)

Query it helps: Query 4

Why we chose it: Season is used as the main filter in the top-scorers query and in many season-scoped features. We indexed it expecting the optimizer to drive the join from the filtered games. It is available to the planner even though, for the 2024 season, the planner chose a different path (discussed in the analysis).

Index 4: Club (name)

```
CREATE INDEX idx_club_name ON Club(name);
```

Table and columns: Club (name)

Query it helps: Query 3

Why we chose it: The club dashboard filters by club name. Indexing “name” lets the query start from the single requested club instead of scanning all clubs and range-checking each one against the OR join condition.

4. Performance After Indexing

After creating the indexes we ran the same five queries again, on the same data and with the same conditions.

Query 1: after indexing

Runtime after indexing: 10.87 s

Query plan (EXPLAIN): Player now uses idx_player_position_dob with type = range (Using index condition), narrowing the scan to ~37,700 rows before the join. The temporary table and filesort remain for the GROUP BY and ORDER BY.

Did the DBMS use the index? Yes

Result: Plan improved, time about the same

Query 2: after indexing

Runtime after indexing: 0.0005 s

Query plan (EXPLAIN): Player uses idx_player_name with type = ref (ref = const), locating the single matching row directly (Using index) before joining to that player's valuations.

Did the DBMS use the index? Yes

Result: Improved (about 22x faster)

Query 3: after indexing

Runtime after indexing: 0.12 s

Query plan (EXPLAIN): Club is now the driving table using idx_club_name with type = ref (ref = const, 1 row). The plan starts from the single Manchester City row, then finds its games, removing the per-record range check.

Did the DBMS use the index? Yes

Result: Improved (about 5x faster)

Query 4: after indexing

Runtime after indexing: 12.81 s

Query plan (EXPLAIN): The plan is essentially unchanged. idx_game_season appears in possible_keys but is not chosen; the optimizer still drives from Club and applies the season filter through the eq_ref join to Game.

Did the DBMS use the index? No

Result: No meaningful change

Query 5: after indexing

Runtime after indexing: 50.31 s

Query plan (EXPLAIN): Player now uses idx_player_position_dob with type = index (Using index), so the position values come straight from the index. The join into the 1.89M-row Appearance table is unchanged.

Did the DBMS use the index? Yes

Result: Small improvement (about 13% faster)

5. Performance Summary Table

Query	Main Tables	Index Created	Before	After	Index Used?	Result
Q1	Player, Appearance	idx_player_position_dob	8.41 s	10.87 s	Yes	Plan improved, time similar
Q2	PlayerValuation, Player	idx_player_name	0.011 s	0.0005 s	Yes	Improved (~22x)
Q3	Club, Game, Competition	idx_club_name	0.65 s	0.12 s	Yes	Improved (~5x)
Q4	Player, Appearance, Game, Club	idx_game_season	13.61 s	12.81 s	No	No change
Q5	Player, Appearance	idx_player_position_dob	57.60 s	50.31 s	Yes	Small improvement (~13%)

All runtimes measured on the fully loaded database (Appearance = 1,894,348 rows). Query numbers here follow the order selected for this checkpoint.

6. Analysis and Explanation

The results show a range of outcomes, which is what we expected once we looked closely at what each query actually spends its time on.

The index that helped the most was `idx_player_name` on Query 2. Looking a player up by name went from a full scan of about 50,000 rows to a single indexed lookup, roughly a 22x speed-up, and the plan changed from `type = ALL` to `type = ref`. The next best was `idx_club_name` on Query 3: indexing the club name let the optimizer start the join from the one club the user asked about instead of scanning all 796 clubs and range-checking each one, giving about a 5x improvement. Both of these are equality lookups on a previously unindexed column, which is the situation where an index helps most.

The indexes that helped least were the ones on the two heaviest queries, Q1 and Q5. In both cases the index did change the plan for the better (Q1's Player access became a range scan, Q5's became an index scan), but the runtime barely moved. The reason is that these queries spend almost all of their time joining and aggregating across the 1.89-million-row Appearance table, and an index on the Player table cannot reduce that work. The bottleneck is the size of the scan on the other side of the join, not the Player lookup we sped up.

Query 4 is the case where MySQL did not use the index we expected. We added `idx_game_season` hoping the query would start from the filtered games, but season 2024 matches several thousand of the roughly 89,000 games, so it is not selective enough for the optimizer to prefer it over the existing join path. The index still appears in the plan's list of candidate keys; the optimizer simply judged another plan cheaper. This is a good reminder that creating an index makes it available, but the optimizer decides whether using it is actually the fastest option based on how selective the filter is.

So the pattern is consistent: indexes gave large gains on selective equality lookups (Q2, Q3), small gains where they only trimmed the cheaper side of a big join (Q1, Q5), and no gain where the filter was not selective enough to change the plan (Q4). In every case the behavior matches what the query plan predicts, rather than being random.

7. Testing Notes

- DBMS: MySQL
- Largest table: Appearance, with 1,894,348 rows. Other large tables include PlayerValuation (507,815) and Game (88,958).
- How runtime was measured: MySQL query profiling (SET profiling = 1, then reading the duration from SHOW PROFILES). Each query was run three times and we reported a representative runtime.
- Plan analysis: EXPLAIN was used to capture the query plan before and after indexing, so we could see the access type, chosen key, estimated rows, and whether a temporary table or filesort was used.
- Limitations: runtimes vary between runs because of caching and buffer-pool state, so small differences (a few percent) should be read as noise rather than a real change. The very fast queries (Q2, Q3) are measured in fractions of a second, where timing overhead is a larger share of the total, so we relied on the change in the query plan as much as on the raw time.

8. Updates to Project Design or Management

No major changes to the project design or management plan at this time. The four indexes above have been added to the database schema and will ship with the final code package.

9. AI-Use Statement

- Our team used Claude (Anthropic) as an AI tool for this checkpoint.
- It was used to help verify the selected queries and EXPLAIN plans, and choose which indexes to test.
- We reviewed each index choice and each result ourselves, including the cases where an index did not help, so that we can explain why the outcome makes sense rather than just reporting numbers.
- We verified the work by running the same queries on the same data before and after indexing under matching conditions, and by checking that the EXPLAIN output agreed with the measured runtimes.